

Lab Test 1 Preparation Guide: Comprehensive Topic Summary & Code Walkthrough

This document serves as a master revision guide for **Lab Test 1**, synthesizing the theoretical principles, architectural mechanisms, and code implementations covered across **Lab 1 through Lab 7** in the 8086 Microprocessor & Assembly Language laboratory.

Table of Contents

1. [Master Lab Reference & Algorithm Mapping Table](#)
2. [Lab-by-Lab Deep Dive & Code Walkthroughs](#)
 - [2.1 Lab 1: Environment Setup & Direct String Display](#)
 - [2.2 Lab 2: Character I/O, Formatting & Multi-Input Staging](#)
 - [2.2.1 Character Echo & Whitespace / Newline Sequences](#)
 - [2.2.2 Staged Multi-Input Buffering & Prompt Interleaving](#)
 - [2.3 Lab 3: Single-Digit Arithmetic & Multi-Digit Output Decomposition](#)
 - [2.3.1 Fundamental 4-Operation Arithmetic](#)
 - [2.3.2 Multi-Digit Output Decomposition via Radix-10 Division](#)
 - [2.4 Lab 4: Compound Academic Marks Calculation & Averages](#)
 - [2.5 Lab 5: Bitwise Operations & Hexadecimal Conversion Subroutines](#)
 - [2.6 Lab 6: Character Case Conversion](#)
 - [2.7 Lab 7: Loops, Conditional Filtering & 1D Array Processing](#)

- [2.7.1 Hardware Loops & Alphabet Traversal](#)
- [2.7.2 Conditional Element Filtering & Jump-Based Loops](#)
- [2.7.3 1D Array Declaration & Pointer Traversal](#)
- [2.7.4 Linear Array Summation & Accumulator Patterns](#)

3. [High-Yield Exam Cheat Sheet & Architecture Mechanics](#)

- [3.1 DOS Interrupt 21h Service Summary](#)
- [3.2 8086 Hardware Division Rules](#)
- [3.3 ASCII & Radix Conversion Rules](#)
- [3.4 Top 5 Common Exam Bugs & Pitfalls](#)

1. Master Lab Reference & Algorithm Mapping Table

The table below outlines each lab, the specific files implemented, the primary algorithms employed, and the foundational 8086 hardware and software concepts required.

Lab	Source File(s)	Primary Algorithm / Task	Core Concepts & Prerequisites Required	Key Registers & Services	Common Exam Pitfalls
Lab 1	1.1.asm lab-1.asm (Identical)	Direct String Display Load address of a string in memory and print it to standard output.	- Memory segmentation (<code>.DATA</code> , <code>.CODE</code> , <code>.STACK</code>) - Initializing <code>DS</code> through accumulator <code>AX</code> - Effective offset calculation (<code>OFFSET</code>) - String	<code>DS</code> , <code>AX</code> <code>DX</code> (string pointer) <code>INT 21H / AH=09H</code> <code>INT 21H / AH=4CH</code>	Forgetting <code>mov ds, ax</code> , resulting in garbage data or null pointer display. Omitting <code>\$</code> at string tail.

Lab	Source File(s)	Primary Algorithm / Task	Core Concepts & Prerequisites Required	Key Registers & Services	Common Exam Pitfalls
			termination using \$		
Lab 2	2.1.asm 2.2.asm 2.3.asm 2.3-alt.asm	Character I/O & Formatting Echoing single/multiple characters with formatted whitespace, CR (13), and LF (10).	- Console character input & output interrupts - ASCII control characters (10 = LF , 13 = CR , 32 = Space) - Staging multiple user inputs into separate 8-bit registers (BL , BH)	AL (input receiver) DL (output carrier) BL , BH (staging) INT 21H / AH=01H INT 21H / AH=02H	Forgetting carriage return (13) alongside line feed (10), causing diagonal cursor stair-stepping. Overwriting AL before saving previous input.
Lab 3	3.1.asm 3.2.asm	Single-Digit Arithmetic & 2-Digit Decimal Decomposition Perform + , - , * , / and split two-digit results into tens and units digits.	- ASCII to integer conversion (- 48) - Integer to ASCII conversion (+ 48) - Hardware multiplication (MUL) & division (DIV) register implicit destinations - Radix-10 decomposition via DIV 10	AL (multiplier/dividend) AH (remainder receiver) AX (16-bit product / dividend) INT 21H / AH=02H	Performing arithmetic directly on raw ASCII characters. Failing to isolate remainder (AH) and quotient (AL) after division.
Lab 4	4.1_CP.asm	Compound Marks Accumulation & Average Calculation Take 3 course grades, compute total marks, format 2-digit sum,	- Sequential variable allocation in .DATA - Multi-operand accumulation - Clearing high byte AH=0 before 8-bit division to	AL (sum & dividend) AH (zeroed out, then remainder) BL , BH (divisors) INT 21H / AH=09H, 01H, 02H	Leaving uninitialized garbage in AH before running DIV reg8 , which triggers CPU Divide Error

Lab	Source File(s)	Primary Algorithm / Task	Core Concepts & Prerequisites Required	Key Registers & Services	Common Exam Pitfalls
		and compute integer average.	prevent divide overflow exceptions • Interleaved user prompts		interrupt (Fault #DE).
Lab 5	5.1.asm	Bitwise Manipulation & 2-Digit Hexadecimal Subroutine Apply bitwise <code>OR</code> , separate upper and lower nibbles, and convert to ASCII hex characters ('0'-'9', 'A'-'F').	• Logical operations (<code>OR</code>, <code>AND</code>, <code>SHR</code>) • Nibble isolation (shift upper nibble by 4; mask lower nibble with <code>0Fh</code>) • Subroutine modularity (<code>CALL</code>, <code>RET</code>, stack tracking) • Conditional ASCII hex adjustment (<code>+48</code>, <code>CMP 57</code>, <code>JBE</code>, <code>+7</code>)	<code>AL</code> (data byte / nibble) <code>BL</code> (preservation register) <code>DL</code> (output carrier) <code>FLAGS</code> (CF, ZF from <code>CMP</code>)	Missing the <code>+7</code> offset for hex digits <code>10-15</code> (<code>0Ah-0Fh</code> → → <code>'A'-'F'</code>). Forgetting <code>RET</code> in <code>PROC</code> , causing the CPU to execute subsequent memory fall-through.
Lab 6	6.1_CP.asm	Character Case Conversion Read a lowercase ASCII character and convert it to uppercase.	• ASCII binary encoding scheme (bit 5 determines case: <code>'a' = 61h</code>, <code>'A' = 41h</code>) • Subtractive arithmetic conversion (<code>SUB AL, 20h</code>) or bitwise masking (<code>AND AL, 0DFh</code>) • Console I/O sequencing	<code>AL</code> (input and conversion) <code>DL</code> (output) <code>INT 21H</code> / <code>AH=01H</code> , <code>02H</code> , <code>09H</code>	Modifying character without validating bounds, or confusing whether to <code>ADD 20h</code> (upper to lower) or <code>SUB 20h</code> (lower to upper).
Lab 7	7.1_loop.asm 7.2_task.asm 7.3_Array.asm	Hardware Loops, Conditional	• Dedicated counter register	<code>CX</code> (loop counter) <code>SI</code> (memory source)	Placing <code>INC SI</code> or <code>INC DL</code> inside

Lab	Source File(s)	Primary Algorithm / Task	Core Concepts & Prerequisites Required	Key Registers & Services	Common Exam Pitfalls
	7.4 Array sum.asm array_Jannat[7.3].asm array_sum_Jannat[7.4].asm array_sum_Semim[7.4].asm	Filtering & 1D Array Operations Print alphabet sequences, skip specific characters, declare 1D byte arrays, traverse with pointers, and calculate vector sums.	<code>CX</code> & hardware <code>LOOP</code> instruction (<code>CX <- CX - 1;</code> <code>JNZ</code>) • Conditional branching (<code>CMP</code> , <code>JE</code> , <code>JBE</code>) • 1D memory buffers (<code>array</code> <code>db ...</code>) • Base indirect pointer indexing using Source Index (<code>SI</code> , <code>[SI]</code>) • Pointer advancement (<code>INC SI</code>)	index pointer) <code>DL</code> (current element output) <code>BL</code> (running sum accumulator) <code>AL</code> (memory transfer mediator)	a skipped block, causing an infinite loop. Adding memory directly to accumulator with unaligned registers.

2. Lab-by-Lab Deep Dive & Code Walkthroughs

2.1 Lab 1: Environment Setup & Direct String Display

- **Files Analyzed:** [1.1.asm](#) and [lab-1.asm](#) (*Identical implementations; merged into one*).
- **Core Problem:** Setting up the standard 8086 segment architecture, allocating memory, and printing a predefined text string to the console.

Key Mechanics & Architectural Insights

- **Memory Segmentation & Directives:**
 - `.model small` : Configures the memory model where code fits in one 64 KB segment and data fits in one 64 KB segment.

- `.stack 100h` : Allocates 256 bytes ($100_{16} = 256_{10}$) for the CPU stack segment (`SS:SP`).

- **The Segment Register Rule:**

- The 8086 architecture does not allow immediate values to be loaded directly into segment registers (e.g., `mov ds, @data` is an illegal opcode).
- Therefore, `@data` is transferred into general accumulator `AX` first, then copied into `DS` :

```
mov ax, @data
mov ds, ax
```

- **DOS Function 09h (`INT 21H`):**

- Requires `AH = 09h` .
- Requires `DX` to hold the offset address of the string buffer (`mov dx, offset myname`).
- Expects the string in memory to terminate with the sentinel symbol `$` (ASCII 24h). Without `$` , the service continues outputting adjacent memory contents until it happens to encounter a `$` byte.

- **DOS Function 4Ch (`INT 21H`):**

- Gracefully returns control to the operating system / DOS environment (`mov ah, 4ch; int 21h`).

```
; =====
; Lab 1: Standard String Display Program
; Files: 1.1.asm / lab-1.asm
; =====

.model small
.stack 100h

.data
    myname db 'my name is Gaus$'    ; '$' indicates end of string
```

```
.code
main proc
    mov ax, @data                ; Initialize data segment
    mov ds, ax

    mov dx, offset myname        ; Load effective address of string
    mov ah, 09h                  ; DOS print string function
    int 21h                      ; Trigger interrupt

exit:
    mov ah, 4ch                  ; Return to DOS
    int 21h

main endp
end main
```

2.2 Lab 2: Character I/O, Formatting & Multi-Input Staging

2.2.1 Character Echo & Whitespace / Newline Sequences

- **Files Analyzed:** [2.1.asm](#) and [2.2.asm](#).
- **Core Problem:** Reading a single character from the keyboard, generating custom spacing and line breaks, and displaying the character back to the screen.

Key Mechanics & Architectural Insights

- **DOS Function 01h (Character Input with Echo):**
 - Reads a single ASCII character from `STDIN`. The character's 8-bit ASCII value is automatically placed into `AL`.
 - To prevent `AL` from being overwritten by subsequent operations, its content is immediately preserved in `BL` (`mov bl, al`).

- **DOS Function 02h (Character Output):**

- Outputs the single byte stored in the `DL` register to `STDOUT`.

- **Standard Line Break Anatomy (CRLF):**

- In DOS/x86 console environments, a true new line requires two distinct operations:

1. **Line Feed (`LF = 10` / `0Ah`):** Moves the cursor straight down to the next physical row without changing the column position.

2. **Carriage Return (`CR = 13` / `0Dh`):** Moves the cursor horizontally back to the leftmost column of the current row.

- Omitting `13` produces a "staircase effect" where the next line starts directly beneath where the previous line finished.

- **Space Character:** ASCII `32` (`20h`) loaded into `DL` outputs a single blank space.

```
; =====  
; Lab 2.1 & 2.2: Character Input, Spacing, and CRLF Output  
; Files: 2.1.asm / 2.2.asm  
; =====  
  
.model small  
.stack 100h  
.code  
main proc  
    ; 1. Read character from keyboard into AL  
    mov ah, 1  
    int 21h  
    mov bl, al                ; Stash input character into BL  
  
    ; 2. Print 4 spaces (ASCII 32)  
    mov ah, 2  
    mov dl, 32
```

```

int 21h

int 21h

int 21h

int 21h


; 3. Print CRLF (New Line + Carriage Return)
mov ah, 2
mov dl, 10          ; Line Feed (move down 1 row)
int 21h
mov dl, 13          ; Carriage Return (move to column 0)
int 21h


; 4. Output the preserved character
mov dl, bl
mov ah, 2
int 21h


exit:
    mov ah, 4ch
    int 21h
main endp
end main

```

2.2.2 Staged Multi-Input Buffering & Prompt Interleaving

- **Files Analyzed:** [2.3.asm](#) and [2.3-alt.asm](#) (Alternative attempted embedding newlines inside `.DATA` strings).
- **Core Problem:** Interleaving text prompts with user inputs, preserving multiple distinct inputs across registers, and echoing them in a formatted output block.

Key Mechanics & Architectural Insights

- **Register Buffering:**

- The first input character is read into `AL` and saved to `BL`.
- The second input character is read into `AL` and saved to `BH`.
- Using `BX` as two distinct 8-bit registers (`BL` and `BH`) avoids allocating unnecessary memory variables.

- **Data Definition Formatting Trap (`2.3-alt.asm`):**

- In `2.3-alt.asm`, an attempt was made to embed carriage returns and multiple string segments directly in the `db` declaration without explicit commas or quotes:

```
; Faulty syntax in 2.3-alt.asm:
var1 db 'Input:$
        5$
        3$'
```

- Standard MASM requires newline bytes embedded within strings to be declared as explicit numeric byte constants:

```
var1 db 'Input:', 10, 13, '$'
```

- The manual CRLF interrupt sequence (`mov dl, 10; int 21h; mov dl, 13; int 21h`) remains the most portable, predictable approach for console formatting.

```
; =====
; Lab 2.3: Staged Multi-Character Input and Formatted Output
; File: 2.3.asm
; =====

.model small
```

```
.stack 100h

.data
    var1 db 'Input:$'
    var2 db 'Output:$'

.code
main proc
    mov ax, @data
    mov ds, ax

    ; Print "Input:"
    mov dx, offset var1
    mov ah, 09h
    int 21h

    ; Print CRLF
    mov ah, 2
    mov dl, 10
    int 21h
    mov dl, 13
    int 21h

    ; Read First Digit -> BL
    mov ah, 1
    int 21h
    mov bl, al

    ; Print CRLF
    mov ah, 2
    mov dl, 10
    int 21h
```

```
mov dl, 13
int 21h

; Read Second Digit -> BH
mov ah, 1
int 21h
mov bh, al

; Print 2x CRLF before Output Header
mov ah, 2
mov dl, 10
int 21h
mov dl, 10
int 21h
mov dl, 13
int 21h

; Print "Output:"
mov dx, offset var2
mov ah, 09h
int 21h

; Print CRLF
mov ah, 2
mov dl, 10
int 21h
mov dl, 13
int 21h

; Echo First Character (BL)
mov ah, 2
```

```
    mov dl, bl
    int 21h

; Print CRLF
    mov ah, 2
    mov dl, 10
    int 21h
    mov dl, 13
    int 21h

; Echo Second Character (BH)
    mov ah, 2
    mov dl, bh
    int 21h

exit:
    mov ah, 4ch
    int 21h
main endp
end main
```

2.3 Lab 3: Single-Digit Arithmetic & Multi-Digit Output Decomposition

2.3.1 Fundamental 4-Operation Arithmetic

- **Files Analyzed:** [3.1.asm](#) .
- **Core Problem:** Read two single-digit numeric inputs from standard input, execute addition, subtraction, multiplication, and division, and output each result separated by spaces.

Key Mechanics & Architectural Insights

- **The ASCII Conversion Barrier:**

- The keyboard interrupt returns ASCII glyph codes, not numerical quantities:

$$\text{ASCII '5'} = 35_{16} = 53_{10}$$

$$\text{ASCII '5'} = 35_{16} = 53_{10}$$

$$\text{ASCII '3'} = 33_{16} = 51_{10}$$

$$\text{ASCII '3'} = 33_{16} = 51_{10}$$

- If added directly without conversion: $53 + 51 = 104 = 'h'$ $53 + 51 = 104 = 'h'$.
- **Conversion to Integer:** Subtract 48 (`30h`):

```
sub al, 48          ; '5' (53) - 48 = 5
```

- **Conversion back to ASCII:** Add 48 (`30h`):

```
add al, 48          ; 5 + 48 = 53 ('5')
```

- **Hardware Multiplication (`MUL`):**

- In 8-bit mode (`mul operand8`), the CPU uses `AL` as an implicit multiplicand:

$$AX \leftarrow AL \times \text{operand8}$$

$$AX \leftarrow AL \times \text{operand8}$$

- For single-digit operations where the product is $< 10 < 10$, the answer is entirely held in `AL`.

- **Hardware Division (`DIV`):**

- In 8-bit mode (`div operand8`), the dividend is **always** assumed to be the 16-bit word in `AX`:

$$AL \text{ (Quotient)} \leftarrow AX \div \text{operand8}$$

$$AL \text{ (Quotient)} \leftarrow AX \div \text{operand8}$$

$$AH \text{ (Remainder)} \leftarrow AX \pmod{\text{operand8}}$$

$$AH \text{ (Remainder)} \leftarrow AX \pmod{\text{operand8}}$$

- Both **AL** and **AH** can then be individually converted back to ASCII (`add al, 48; add ah, 48`) to display the quotient and remainder separately.

```
; =====  
; Lab 3.1: Fundamental Arithmetic Operations (+, -, *, /)  
; File: 3.1.asm  
; =====  
  
.model small  
.stack 100h  
.data  
    num1  db ?  
    num2  db ?  
    sum   db ?  
    sub   db ?  
    multi db ?  
    divi  db ?  
    rem   db ?  
  
.code  
main proc  
    mov ax, @data  
    mov ds, ax  
  
    ; Read First Number  
    mov ah, 1  
    int 21h  
    sub al, 48          ; Convert ASCII -> Integer  
    mov num1, al  
  
    ; Space delimiter
```

```
mov ah, 2
mov dl, 32
int 21h

; Read Second Number
mov ah, 1
int 21h
sub al, 48          ; Convert ASCII -> Integer
mov num2, al

; Space delimiter
mov ah, 2
mov dl, 32
int 21h

; 1. ADDITION
mov al, num1
add al, num2
add al, 48          ; Convert Integer -> ASCII
mov sum, al
mov dl, sum
mov ah, 2
int 21h

; 2. SUBTRACTION
mov ah, 2
mov dl, 32
int 21h
mov al, num1
sub al, num2
add al, 48
```

```
mov subtr, al
mov dl, subtr
mov ah, 2
int 21h
```

```
; 3. MULTIPLICATION
```

```
mov ah, 2
mov dl, 32
int 21h
mov al, num1
mul num2           ; AX = AL * num2
add al, 48
mov multi, al
mov dl, multi
mov ah, 2
int 21h
```

```
; 4. DIVISION
```

```
mov ah, 2
mov dl, 32
int 21h
mov al, num1
mov ah, 0           ; Clear upper byte for clean division
div num2            ; AL = Quotient, AH = Remainder
add al, 48
add ah, 48
mov divi, al
mov rem, ah
```

```
; Print Quotient
```

```
mov dl, divi
```

```

mov ah, 2
int 21h

; Print Space
mov dl, 32
mov ah, 2
int 21h

; Print Remainder
mov dl, rem
mov ah, 2
int 21h

main endp
end main

```

2.3.2 Multi-Digit Output Decomposition via Radix-10 Division

- **Files Analyzed:** [3.2.asm](#).
- **Core Problem:** If multiplication yields a product ≥ 10 (e.g., $4 \times 3 = 12$), adding 48 directly produces character code 6060 (<), corrupting the output. The numerical value must be decomposed into individual decimal digits.

Key Mechanics & Architectural Insights

- **The Decimal Decomposition Algorithm:**
 - Any 2-digit decimal number $N \in [0, 99]$ can be separated by dividing by 10:

$$\text{Tens Digit} = \lfloor N/10 \rfloor$$

$$\text{Tens Digit} = [N/10]$$

$$\text{Units Digit} = N \pmod{10}$$

$$\text{Units Digit} = N \pmod{10}$$

◦ In 8086 assembly, setting `BL = 10` and running `div bl` on `AX` ($AH = 0, AL = NAH = 0, AL = N$) produces:

- `AL` = Quotient (tens digit)
- `AH` = Remainder (units digit)

• Sequential Output:

- Add 48 to `AL` → → print with `AH=02h` (outputs the tens digit first).
- Add 48 to `AH` → → print with `AH=02h` (outputs the units digit second).

```
; =====  
; Lab 3.2: Decomposing a 2-Digit Product via Base-10 Division  
; File: 3.2.asm  
; =====  
  
.model small  
.stack 100h  
.data  
    num1    db ?  
    num2    db ?  
    multi   db ?  
    quot    db ?  
    rem     db ?  
  
.code  
main proc  
    mov ax, @data  
    mov ds, ax  
  
    ; Read Inputs num1 and num2  
    mov ah, 1  
    int 21h
```

```

sub al, 48
mov num1, al

mov ah, 1
int 21h
sub al, 48
mov num2, al

; Multiply: AL = num1 * num2
mov al, num1
mul num2                ; AX = AL * num2 (e.g., 4 * 3 = 12)
mov multi, al

; Decompose Decimal Digits
mov ah, 0                ; Ensure AH is 0 before 8-bit division
mov bl, 10
div bl                   ; AL = 12 / 10 = 1 (tens), AH = 12 % 10 = 2 (units)

add al, 48                ; 1 + 48 = '1'
add ah, 48                ; 2 + 48 = '2'
mov quot, al
mov rem, ah

; Output Tens Digit
mov ah, 2
mov dl, quot
int 21h

; Output Units Digit
mov ah, 2
mov dl, rem

```

```
int 21h

main endp
end main
```

2.4 Lab 4: Compound Academic Marks Calculation & Averages

- **Files Analyzed:** [4.1_CP.asm](#) .
- **Core Problem:** Prompt user for 3 subject marks, calculate the total sum, display the 2-digit total marks, and calculate/display the integer average mark.

Key Mechanics & Architectural Insights

- **The "Garbage AH" Trap (Fault #DE):**
 - When preparing to divide by a byte register (`div bl`), the CPU divides the entire 16-bit register `AX` (`AH:AL`), **not** just `AL` .
 - If `AH` contains leftover values from earlier DOS calls (such as `AH = 02h` from printing), the CPU evaluates a dividend of:
$$AX = (AH \times 256) + AL$$
$$AX = (AH \times 256) + AL$$
 - For example, if `AH = 2` and `AL = 27` , $AX = 512 + 27 = 539$ $AX = 512 + 27 = 539$.
 - Dividing 539539 by 33 gives 179179. Since 179179 cannot fit into an 8-bit signed/unsigned register target if scaled, or if the quotient exceeds 255, the CPU immediately raises **Interrupt 0 (Divide Error Exception)**.
 - **The Golden Rule:** Always explicitly clear `AH` before an 8-bit division:

```
mov ah, 0          ; Zero-extend AL into AX
div bl
```

- **Summation & Multi-Digit Formatting:**

- Accumulate values in `AL` :

```
mov al, num1
add al, num2
add al, num3
mov sum, al
```

- Decompose into two decimal digits using `div 10` .
- Calculate average using integer division: $\text{Average} = \text{Sum}/3$

```
; =====
; Lab 4: Subject Marks Summation & Average Calculation
; File: 4.1_CP.asm
; =====

.model small
.stack 100h
.data
    num1  db ?
    num2  db ?
    num3  db ?
    sum   db ?
    quot  db ?
    rem   db ?
    avg_q db ?
    avg_r db ?
    msg1  db "Subject 1: $"
    msg2  db "Subject 2: $"
    msg3  db "Subject 3: $"
    msg4  db "Total Marks: $"
```

```
msg5 db "Average Marks: $"
```

```
.code
```

```
main proc
```

```
    mov ax, @data
```

```
    mov ds, ax
```

```
    ; Read Subject 1
```

```
    mov dx, offset msg1
```

```
    mov ah, 09h
```

```
    int 21h
```

```
    mov ah, 1
```

```
    int 21h
```

```
    sub al, 48
```

```
    mov num1, al
```

```
    ; Print CRLF
```

```
    mov ah, 2
```

```
    mov dl, 10
```

```
    int 21h
```

```
    mov dl, 13
```

```
    int 21h
```

```
    ; Read Subject 2
```

```
    mov dx, offset msg2
```

```
    mov ah, 09h
```

```
    int 21h
```

```
    mov ah, 1
```

```
    int 21h
```

```
    sub al, 48
```

```
    mov num2, al
```

```
; Print CRLF
mov ah, 2
mov dl, 10
int 21h
mov dl, 13
int 21h

; Read Subject 3
mov dx, offset msg3
mov ah, 09h
int 21h
mov ah, 1
int 21h
sub al, 48
mov num3, al

; Compute Sum: AL = num1 + num2 + num3
mov al, num1
add al, num2
add al, num3
mov sum, al

; Print CRLF
mov ah, 2
mov dl, 10
int 21h
mov dl, 13
int 21h

; Display "Total Marks: "
```

```

mov dx, offset msg4
mov ah, 09h
int 21h

; Display 2-digit Total
mov al, sum
mov ah, 0                ; CRITICAL: zero out AH before division
mov bl, 10
div bl                   ; AL = tens, AH = units
add al, 48
add ah, 48
mov quot, al
mov rem, ah

mov ah, 2
mov dl, quot
int 21h
mov dl, rem
int 21h

; Print CRLF
mov ah, 2
mov dl, 10
int 21h
mov dl, 13
int 21h

; Display "Average Marks: "
mov dx, offset msg5
mov ah, 09h
int 21h

```

```

    ; Compute Average: Sum / 3
    mov al, sum
    mov ah, 0                ; CRITICAL: zero out AH again
    mov bl, 3
    div bl                   ; AL = integer average
    add al, 48
    mov avg_q, al

    mov ah, 2
    mov dl, avg_q
    int 21h

exit:
    mov ah, 4ch
    int 21h
main endp
end main

```

2.5 Lab 5: Bitwise Operations & Hexadecimal Conversion Subroutines

- **Files Analyzed:** `5.1.asm`.
- **Core Problem:** Perform bitwise logical manipulation (`OR`) on a data byte, isolate the upper and lower 4-bit nibbles, and convert them to human-readable ASCII hexadecimal digits ('0'-'9', 'A'-'F') using a reusable procedure (`PROC`).

Key Mechanics & Architectural Insights

- **Bitwise Logic:**
 - `or al, 0D1h` : Sets specific bit positions while retaining existing ones.

- **Nibble Isolation Mechanics:**

- An 8-bit byte consists of two 4-bit nibbles: `[Bit 7..4]` (Upper) and `[Bit 3..0]` (Lower).
- **Upper Nibble:** Extracted by shifting right 4 bits (`shr al, 4`), which slides bits 7..4 down to bit positions 3..0 and fills bits 7..4 with zeros.
- **Lower Nibble:** Extracted by bitwise masking with `0Fh` (`00001111b`): `and al, 0Fh` .

- **Hexadecimal ASCII Conversion Math:**

- For values $0 \leq X \leq 9$: Adding 48 converts the value directly to `'0'` through `'9'` .
- For values $10 \leq X \leq 15$:
 - Adding 48 yields values from 58 to 63 (`:` , `;` , `<` , `=` , `>` , `?`).
 - ASCII `'A'` is 65₁₀.
 - Therefore, an additional offset of 7 ($65 - 58 = 7$) must be added:

$$\text{Hex ASCII} = X + 48 + 7 = X + 55$$

$$\text{Hex ASCII} = X + 48 + 7 = X + 55$$

- **Subroutines with `CALL` and `RET` :**

- `call print_hex` : Pushes the current Instruction Pointer (`IP`) onto the stack (`SP <- SP - 2`) and jumps to the procedure label.
- `ret` : Pops the saved address from the stack back into `IP` , resuming execution immediately after the `CALL` instruction.

```
+-----+
|           Hexadecimal Conversion Logic Flow           |
+-----+

      |
      | AL = Nibble (0-15)
      |
      |
      | add al, 48
```

```

        |
        cmp al, 57 ('9')
        |
        +-----+-----+
        |                                     |
AL <= 57 (0-9)      AL > 57 (10-15)
        |                                     |
        JBE                                     add al, 7
        |                                     |
        +----->+<-----+
        |
        mov dl, al
        mov ah, 02h
        int 21h
        |
        RET

```

```

; =====
; Lab 5: Bitwise Manipulation & Hexadecimal Subroutine
; File: 5.1.asm
; =====

.model small
.stack 100h
.data
    num1 db 018h
    msg1 db "num1 : $"

.code
main proc
    mov ax, @data

```

```

    mov ds, ax

; Display Prompt
    mov dx, offset msg1
    mov ah, 09h
    int 21h

; Apply Bitwise OR
    mov al, num1
    or al, 0D1h                ; 0001 1000 OR 1101 0001 = 1101 1001 (0D9h)
    mov bl, al                 ; Preserve result in BL

; Process Upper Nibble
    mov al, bl
    shr al, 4                  ; Shift upper 4 bits into lower nibble position
    call print_hex

; Process Lower Nibble
    mov al, bl
    and al, 0Fh                ; Mask out upper 4 bits (0000 1111b)
    call print_hex

; Exit
    mov ah, 4Ch
    int 21h
main endp

; -----
; Procedure: print_hex
; Input: AL (holds single 4-bit nibble value 0..15)
; Output: Prints corresponding hex character ('0'-'9', 'A'-'F')

```

```

; -----
print_hex proc
    add al, 48                ; Convert 0..9 to '0'..'9'
    cmp al, 57                ; Compare with ASCII '9'
    jbe out_label            ; If <= '9', skip letter adjustment
    add al, 7                 ; Adjust 10..15 to 'A'..'F'

out_label:
    mov dl, al
    mov ah, 02h
    int 21h
    ret                      ; Return control back to caller
print_hex endp

end main

```

2.6 Lab 6: Character Case Conversion

- **Files Analyzed:** [6.1_CP.asm](#) .
- **Core Problem:** Read an arbitrary lowercase character from standard input, convert it to its uppercase equivalent, and print the output.

Key Mechanics & Architectural Insights

- **The ASCII Case Bit Property:**
 - In ASCII, the difference between any lowercase letter and its corresponding uppercase letter is exactly $32_{10} = 20_{16}$
 $= 20_{16}$:

'a' = 01100001₂ = 61₁₆

'A' = 01000001₂ = 41₁₆

'a' = 01100001₂ = 61₁₆

'A' = 010000012 = 4116

Bit 5 is the 'case bit' (1 for lowercase, 0 for uppercase).

Bit 5 is the 'case bit' (1 for lowercase, 0 for uppercase).

- **Conversion Techniques:**

- 1. **Arithmetic Subtraction:**

```
sub al, 20h          ; 'a' (61h) - 20h = 'A' (41h)
```

- 2. **Bitwise Masking (Idempotent approach):**

```
and al, 0DFh        ; 0DFh = 11011111b (clears bit 5, forcing uppercase)
```

- **Opposite Conversion (Uppercase to Lowercase):**

- Add 20₁₆2016 (`add al, 20h`) or set bit 5 (`or al, 20h`).

```
; =====  
; Lab 6: Lowercase to Uppercase Case Conversion  
; File: 6.1_CP.asm  
; =====  
  
.model small  
.stack 100h  
.data  
    msg1 db "Input: $"  
    msg2 db "Output : $"  
    char db ?  
  
.code
```

```
main proc
    mov ax, @data
    mov ds, ax

    ; Prompt for Input
    mov dx, offset msg1
    mov ah, 09h
    int 21h

    ; Read Lowercase Character into AL
    mov ah, 01h
    int 21h

    ; Convert to Uppercase
    sub al, 20h                ; Subtract 32 (20h)
    mov char, al

    ; Print CRLF
    mov ah, 02h
    mov dl, 10
    int 21h
    mov dl, 13
    int 21h

    ; Display Output Prompt
    mov dx, offset msg2
    mov ah, 09h
    int 21h

    ; Print Converted Uppercase Character
    mov ah, 02h
```

```

    mov dl, char
    int 21h

exit:
    mov ah, 4ch
    int 21h
main endp
end main

```

2.7 Lab 7: Loops, Conditional Filtering & 1D Array Processing

2.7.1 Hardware Loops & Alphabet Traversal

- **Files Analyzed:** `7.1_loop.asm`.
- **Core Problem:** Sequentially display all 26 lowercase English alphabet characters ('a' through 'z') using the hardware loop counter register.

Key Mechanics & Architectural Insights

- **The 8086 `LOOP` Instruction:**
 - The `LOOP` instruction is hardwired to use the `CX` (Count) register.
 - Every time `loop label` executes, the CPU performs two sequential actions atomically:
 1. Decrements `CX`: $CX \leftarrow CX - 1$
 2. Tests condition: If $CX \neq 0$, branch to `label`; otherwise continue to the next instruction.
 - Note: `LOOP` does **not** modify the Zero Flag (`ZF`) in the `FLAGS` register.
- **Initialization:**
 - To print 26 characters, load `CX` with 26 (`mov cx, 26`).

- Initialize the character carrier in `DL` with `'a'` (`mov dl, 'a'`).
- Advance the character each iteration using `inc dl` .

```
; =====  
; Lab 7.1: Alphabet Traversal using Hardware LOOP  
; File: 7.1_loop.asm  
; =====  
  
.model small  
.stack 100h  
.code  
main proc  
    mov cx, 26                ; Counter for 26 letters  
    mov dl, 'a'              ; Start at 'a'  
  
level:  
    mov ah, 2h                ; Print current character in DL  
    int 21h  
    inc dl                    ; Move to next ASCII letter ('a' -> 'b')  
    loop level                 ; CX = CX - 1; if CX != 0 jump to level  
  
exit:  
    mov ah, 4ch  
    int 21h  
main endp  
end main
```

2.7.2 Conditional Element Filtering & Jump-Based Loops

- **Files Analyzed:** `7.2_task.asm` and `array_sum_Semim[7.4].asm` (which actually contains Semim's upper-case A-Z filtering algorithm skipping 'S').
- **Core Problem:** Iterate through an alphabet sequence while skipping a specific character (e.g., skip letter `'s'` or `'S'`).

Key Mechanics & Architectural Insights

- **Selective Branching with `CMP` and `JE` :**
 - In each iteration, test `DL` against the target skip character: `cmp dl, 's'` .
 - If equal (`je skip`), bypass the `int 21h` character print service.
- **The Counter / Increment Sync Trap:**
 - Ensure `inc dl` and `loop level` are placed **after** or **at** the `skip` label so that:
 1. `DL` still gets incremented to the next letter.
 2. `CX` is decremented properly.
 - If `skip` jumped directly to `loop level` without incrementing `DL` , the loop would compare `'s'` forever, locking into an infinite loop.
- **Alternative: Bound Comparison Loop without `CX` (`array_sum_Semim`):**
 - Rather than relying on `CX` and `LOOP` , you can increment `DL` and compare directly against the terminal boundary (`cmp dl, 'Z'` ; `jbe level`).

```
; =====
; Lab 7.2: Alphabet Loop with Conditional Skip ('s')
; File: 7.2_task.asm
; =====

.model small
.stack 100h
.code
```

```

main proc
    mov cx, 26
    mov dl, 'a'

level:
    cmp dl, 's'                ; Check if current char is 's'
    je skip                    ; If matched, bypass printing

    mov ah, 2h                 ; Print character
    int 21h

skip:
    inc dl                     ; Increment to next letter ('s' -> 't')
    loop level                  ; Decrement CX and repeat

exit:
    mov ah, 4ch
    int 21h
main endp
end main

```

2.7.3 1D Array Declaration & Pointer Traversal

- **Files Analyzed:** [7.3_Array.asm](#) and [array_Jannat\[7.3\].asm](#).
- **Core Problem:** Define a contiguous 1-dimensional array of byte-sized integers in the data segment, load its base address into a pointer index register, and traverse it using indexed addressing to print each element separated by spaces.

Key Mechanics & Architectural Insights

- **Memory Buffer Declaration (`db`):**

```
array db 1, 2, 3, 4, 5
```

Allocates 5 contiguous bytes in memory: `[array+0] = 1` , `[array+1] = 2` , etc.

- **Base Pointer Indexing with Source Index (`SI`):**

- `mov si, offset array` : Loads the starting memory offset of the array into `SI` .
- Dereferencing `[si]` : Retrieves the byte stored at memory address `DS:SI` .

- **Pointer Arithmetic:**

- Because elements are defined as bytes (`db`), the pointer advances by 1 byte per element: `inc si` .
- (If arrays were declared as 16-bit words `dw` , the pointer would need `add si, 2`).

- **Conversion During Display:**

- Array elements stored as raw integers (1, 2, 3...) must have 48 added (`add dl, 48`) before passing to `INT 21H / AH=02H` .

```
; =====  
; Lab 7.3: 1D Array Traversal and Space-Delimited Output  
; File: 7.3_Array.asm  
; =====  
.model small  
.stack 100h  
.data  
    array db 1, 2, 3, 4, 5          ; 5-element byte array  
  
.code  
main proc  
    mov ax, @data  
    mov ds, ax
```

```

    mov si, offset array          ; Load base memory address of array
    mov cx, 5                     ; Array length into loop counter

my_loop:
    mov ah, 2
    mov dl, [si]                  ; Dereference pointer: DL = array[i]
    add dl, 48                    ; Convert integer to ASCII
    int 21h

    ; Print Space delimiter
    mov dl, 32
    int 21h

    inc si                        ; Advance pointer to next byte
    loop my_loop                  ; Repeat for all elements

exit:
    mov ah, 4ch
    int 21h
main endp
end main

```

2.7.4 Linear Array Summation & Accumulator Patterns

- **Files Analyzed:** [7.4_Array_sum.asm](#) , [array_sum_Jannat\[7.4\].asm](#) , and [array_sum_Semim\[7.4\].asm](#) .
- **Core Problem:** Traverse an integer array, sum all its values into an accumulator register, and print the single-digit sum.

Key Mechanics & Critical Bug Analysis

- **Accumulator Initialization:**

- An 8-bit register (such as `BL`) is initialized to zero: `mov bl, 0`.
- **Addressing / Transfer Logic Bug in `7.4_Array_sum.asm`:**
 - In `7.4_Array_sum.asm`, the loop was written as:

```
mov ah, 2
mov dl, [si]
add bl, al      ; BUG: AL was never loaded from [si]! AL contains garbage or leftover @data!
inc si
loop my_loop
```

- Here, the array element was moved to `DL`, but `BL` was accumulating `AL`. This results in completely incorrect sum values.
- **The Correct Pattern (`array_sum_Jannat[7.4].asm`):**
 - Load the memory byte directly into the transfer register `AL`, then add `AL` to accumulator `BL`:

```
mov al, [si]    ; Dereference current array element
add bl, al      ; Accumulate into BL
inc si         ; Advance memory pointer
loop my_loop
```

- *Note on Jannat's variant:* `cx` was set to 5 for a 3-element array (`array db 1,2,3`), which reads 2 bytes of out-of-bounds unallocated memory past the array boundary. The loop count `cx` must strictly match the array's declared size.

```
; =====
; Lab 7.4: Robust 1D Array Accumulation and Sum Output
; Merged & Corrected Reference Implementation
; =====
```

```

.model small
.stack 100h
.data
    array db 1, 2, 3, 4          ; 4-element byte array (Sum = 1+2+3+4 = 10)
    ; For single-digit test: array db 1, 2, 3 (Sum = 6)

.code
main proc
    mov ax, @data
    mov ds, ax

    mov si, offset array        ; SI points to array[0]
    mov cx, 4                   ; Loop counter matches array size
    mov bl, 0                   ; Accumulator cleared to 0

sum_loop:
    mov al, [si]                ; Load current array element into AL
    add bl, al                  ; Accumulate: BL = BL + AL
    inc si                      ; Move pointer to next element
    loop sum_loop              ; CX = CX - 1; repeat until CX == 0

    ; Output single-digit sum (or decompose via DIV 10 if sum >= 10)
    mov dl, bl
    add dl, 48                  ; Convert integer sum to ASCII
    mov ah, 2
    int 21h

exit:
    mov ah, 4ch
    int 21h

```

```
main endp
end main
```

3. High-Yield Exam Cheat Sheet & Architecture Mechanics

3.1 DOS Interrupt 21h Service Summary

Service (AH)	Purpose	Input Arguments	Output / Side Effects
01h	Read Character with Echo	None	AL = ASCII character code read from standard input.
02h	Write Character	DL = ASCII character to display	Writes character to standard output.
09h	Write String	DX = Offset address of \$ - terminated string	Writes characters to standard output until \$ is encountered.
4Ch	Terminate Process	AL = Return exit code (optional)	Relinquishes CPU control cleanly back to DOS environment.

3.2 8086 Hardware Division Rules

Division Type	Divisor Size	Dividend Register	Quotient Destination	Remainder Destination	Overflow Condition
div req8 / div mem8	8-bit (Byte)	AX (16-bit) (Must ensure AH = 0)	AL (8-bit)	AH (8-bit)	Quotient > 255> 255 triggers Divide Error (Fault #DE)
div req16 / div mem16	16-bit (Word)	DX:AX (32-bit) (Must ensure DX = 0)	AX (16-bit)	DX (16-bit)	Quotient > 65535> 65535 triggers Divide Error (Fault #DE)

3.3 ASCII & Radix Conversion Rules

```
+-----+
| ASCII to Int | --> SUB AL, 48 ('0' -> 0)
+-----+
| Int to ASCII | --> ADD AL, 48 (0 -> '0')
+-----+
| Lower to Upper | --> SUB AL, 20h ('a' -> 'A')
+-----+
| Upper to Lower | --> ADD AL, 20h ('A' -> 'a')
+-----+
| Hex Digit (10-15) | --> ADD AL, 48 + 7 ('A'-'F')
+-----+
```

3.4 Top 5 Common Exam Bugs & Pitfalls

1. Uninitialized Data Segment:

- Omitting `mov ax, @data; mov ds, ax` means memory variables and string offsets point to unmapped or invalid physical addresses.

2. Leftover Garbage in `AH` Before `DIV`:

- Leaving `AH` unzeroed before `div bl` causes `AX` to be interpreted as hundreds or thousands, triggering CPU Divide Error (#DE). Always insert `mov ah, 0`.

3. Omitting Carriage Return (`13`):

- Outputting only Line Feed (`10`) drops the cursor vertically without returning to column 0. Always emit both `10` and `13`.

4. Missing Subroutine `RET` :

- A procedure (`proc`) without `ret` falls through into whatever instructions follow sequentially in the code segment.

5. Array Out-of-Bounds in `LOOP` :

- Setting `CX` to a count greater than the declared array size reads unallocated memory bytes, polluting accumulators and arithmetic sums. Ensure `CX` strictly equals the number of elements.